Encode-decode for coproducts: paths out of an injection are equivalent to the code family

The Univalent Foundations Program

hott-book:basics.tex:local-snapshot-20261014; CC BY-SA 3.0 Unported

Editorial note. The counted statement is the author theorem labelled thm:path-coprod (source lines 1982-1984); the proof span 1985-2031 is the author's proof environment, retained line for line.

Editorial note. The proof is a complete encode-decode argument with both quasi-inverse directions displayed; the two citation tags are rewriting tags inside an otherwise self-contained computation chain.

Editorial note. The author's remark after the theorem that a fixed endpoint on the right gives a corresponding theorem is outside the counted span.

Editorial note. Revision history: r001 assembly was refused because the external refs were flagged load-bearing and r002 because their kinds were outside the policy vocabulary; r003 restates the refs as reviewed navigation pointers with policy-vocabulary kinds, without changing the retained spans.

Editorial scope. One original result of the HoTT book chapter cha:basics, counted once: the encode-decode characterization of paths out of an injection. The context retains the author's setup of the code family; the disjointness corollaries drawn after the theorem are not counted.

Allowed standard prerequisites (editorial).

- The coproduct type with its constructors and its elimination principle.
- The empty type and its elimination rule.
- The code family on the coproduct defined by recursion, with the fixed endpoint element (retained context).
- Transport along a path in a type family; based path induction with its conversion rule.
- Equivalence exhibited by quasi-inverses.

Source reference. Original lemma 2.3.10: sources/hott-basics/pinned-original-source.tex, source lines 956–956. Author lemma asserted at source lines 956-959 (book 2.3.10); cited by one rewriting tag inside the retained computation chain. Navigation pointer only: the counted proof is retained verbatim and no part of it is replaced by the reference; the author uses the cited lemma as established.

Source reference. Original lemma 2.11.2: sources/hott-basics/pinned-original-source.tex, source lines 1875–1875. Author corollary proved in place in the

source (lemma environment, book 2.11.2) and retained in-corpus as a sibling card; cited by the following rewriting tag of the same chain. Navigation pointer only: the counted proof is retained verbatim and no part of it is replaced by the reference.

Consider the coproduct type $A + B$, which is "presented" by the injections $\mathsf{inl} : A \to A+B$ and $\mathsf{inr} : B \to A+B$. Intuitively, we expect that $A+B$ contains exact copies of A and B disjointly, so that we should have

$$(\mathsf{inl}(a_1) = \mathsf{inl}(a_2)) \simeq (a_1 = a_2) \tag{0.0.1}$$
$$(\mathsf{inr}(b_1) = \mathsf{inr}(b_2)) \simeq (b_1 = b_2) \tag{0.0.2}$$
$$(\mathsf{inl}(a) = \mathsf{inr}(b)) \simeq \mathbf{0}. \tag{0.0.3}$$

We prove this as follows. Fix an element $a_0 : A$; we will characterize the type family

$$(x \mapsto (\mathsf{inl}(a_0) = x)) : A + B \to \mathcal{U}. \tag{0.0.4}$$

A similar argument would characterize the analogous family $x \mapsto (x = \mathsf{inr}(b_0))$ for any $b_0 : B$. Together, these characterizations imply (0.0.1)–(0.0.3).

In order to characterize (0.0.4), we will define a type family $\mathsf{code} : A + B \to \mathcal{U}$ and show that $\prod_{(x:A+B)}((\mathsf{inl}(a_0) = x) \simeq \mathsf{code}(x))$. Since we want to conclude (0.0.1) from this, we should have $\mathsf{code}(\mathsf{inl}(a)) = (a_0 = a)$, and since we also want to conclude (0.0.3), we should have $\mathsf{code}(\mathsf{inr}(b)) = \mathbf{0}$. The essential insight is that we can use the recursion principle of $A + B$ to *define* $\mathsf{code} : A + B \to \mathcal{U}$ by these two equations:

$$\mathsf{code}(\mathsf{inl}(a)) :\equiv (a_0 = a),$$
$$\mathsf{code}(\mathsf{inr}(b)) :\equiv \mathbf{0}.$$

Theorem 0.0.5. *For all $x : A + B$ we have $(\mathsf{inl}(a_0) = x) \simeq \mathsf{code}(x)$.*

Proof. The key to the following proof is that we do it for all points x together, enabling us to use the elimination principle for the coproduct. We first define a function

$$\mathsf{encode} : \prod_{(x:A+B)} \prod_{(p:\mathsf{inl}(a_0)=x)} \mathsf{code}(x)$$

by transporting reflexivity along p:

$$\mathsf{encode}(x, p) :\equiv \mathsf{transport}^{\mathsf{code}}(p, \mathsf{refl}_{a_0}).$$

Note that $\mathsf{refl}_{a_0} : \mathsf{code}(\mathsf{inl}(a_0))$, since $\mathsf{code}(\mathsf{inl}(a_0)) \equiv (a_0 = a_0)$ by definition of code. Next, we define a function

$$\mathsf{decode} : \prod_{(x:A+B)} \prod_{(c:\mathsf{code}(x))} (\mathsf{inl}(a_0) = x).$$

To define $\mathsf{decode}(x, c)$, we may first use the elimination principle of $A + B$ to divide into cases based on whether x is of the form $\mathsf{inl}(a)$ or the form $\mathsf{inr}(b)$.

In the first case, where $x \equiv \mathsf{inl}(a)$, then $\mathsf{code}(x) \equiv (a_0 = a)$, so that c is an identification between a_0 and a. Thus, $\mathsf{ap}_{\mathsf{inl}}(c) : (\mathsf{inl}(a_0) = \mathsf{inl}(a))$ so we can define this to be $\mathsf{decode}(\mathsf{inl}(a), c)$.

In the second case, where $x \equiv \mathsf{inr}(b)$, then $\mathsf{code}(x) \equiv \mathbf{0}$, so that c inhabits the empty type. Thus, the elimination rule of $\mathbf{0}$ yields a value for $\mathsf{decode}(\mathsf{inr}(b), c)$.

This completes the definition of decode; we now show that $\mathsf{encode}(x, -)$ and $\mathsf{decode}(x, -)$ are quasi-inverses for all x. On the one hand, suppose given $x : A + B$ and $p : \mathsf{inl}(a_0) = x$; we want to show $\mathsf{decode}(x, \mathsf{encode}(x, p)) = p$. But now by (based) path induction, it suffices to consider $x \equiv \mathsf{inl}(a_0)$ and $p \equiv \mathsf{refl}_{\mathsf{inl}(a_0)}$:

$$\begin{aligned}
\mathsf{decode}(x, \mathsf{encode}(x, p)) &\equiv \mathsf{decode}(\mathsf{inl}(a_0), \mathsf{encode}(\mathsf{inl}(a_0), \mathsf{refl}_{\mathsf{inl}(a_0)})) \\
&\equiv \mathsf{decode}(\mathsf{inl}(a_0), \mathsf{transport}^{\mathsf{code}}(\mathsf{refl}_{\mathsf{inl}(a_0)}, \mathsf{refl}_{a_0})) \\
&\equiv \mathsf{decode}(\mathsf{inl}(a_0), \mathsf{refl}_{a_0}) \\
&\equiv \mathsf{ap}_{\mathsf{inl}}(\mathsf{refl}_{a_0}) \\
&\equiv \mathsf{refl}_{\mathsf{inl}(a_0)} \\
&\equiv p.
\end{aligned}$$

On the other hand, let $x : A+B$ and $c : \mathsf{code}(x)$; we want to show $\mathsf{encode}(x, \mathsf{decode}(x, c)) = c$. We may again divide into cases based on x. If $x \equiv \mathsf{inl}(a)$, then $c : a_0 = a$ and $\mathsf{decode}(x, c) \equiv \mathsf{ap}_{\mathsf{inl}}(c)$, so that

$$\begin{aligned}
\mathsf{encode}(x, \mathsf{decode}(x, c)) &\equiv \mathsf{transport}^{\mathsf{code}}(\mathsf{ap}_{\mathsf{inl}}(c), \mathsf{refl}_{a_0}) \\
&= \mathsf{transport}^{a \mapsto (a_0 = a)}(c, \mathsf{refl}_{a_0}) \\
&\qquad \text{(by original lemma 2.3.10)} \\
&= \mathsf{refl}_{a_0} \cdot c \qquad \text{(by original lemma 2.11.2)} \\
&= c.
\end{aligned}$$

Finally, if $x \equiv \mathsf{inr}(b)$, then $c : \mathbf{0}$, so we may conclude anything we wish. □